

TD API (~8H)

L'objectif de ce TD est de développer une API pour exposer vos données. Cette API constituera la partie visible de votre projet pour les utilisateurs. Elle permettra, par exemple, de présenter des métriques liées aux événements collectés. L'API représentera le produit final que vous présenterez lors de la dernière session.

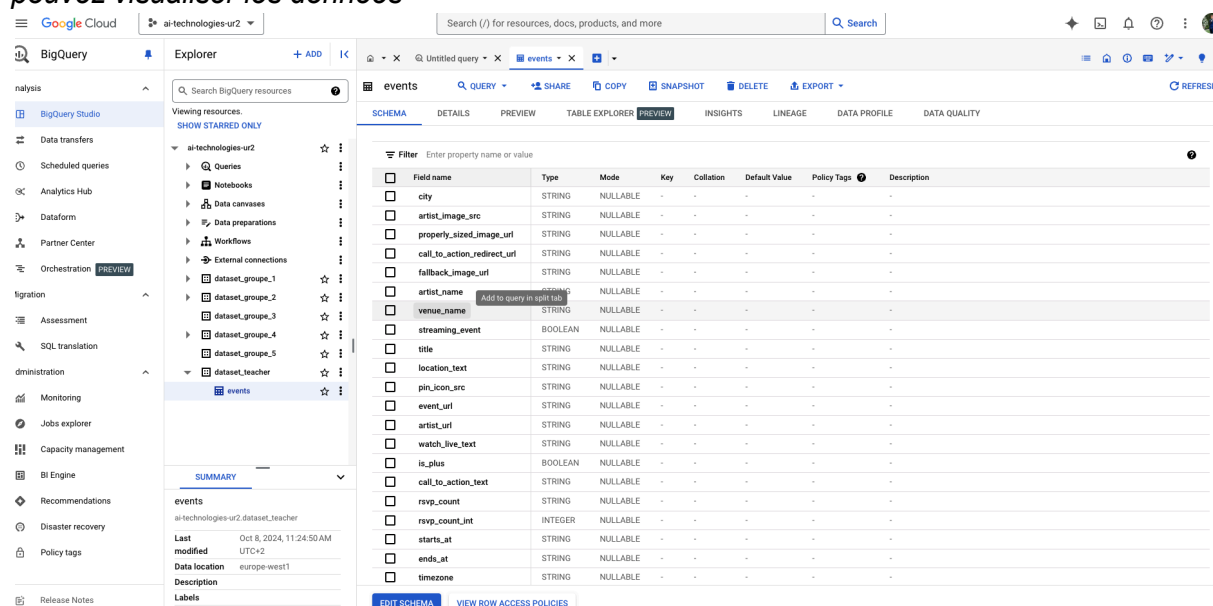
Groupe 1 : Paris
Groupe 2 : New York
Groupe 3 : Londres
Groupe 4 : Berlin
Groupe 5 :

I - Mise en place de l'environnement de travail

1. Vérifier que vous avez bien accès à votre projet GIT

2. Vérifier que vous avez bien accès à BigQuery

Vous devriez avoir uploadé vos données enrichies dans votre dataset BigQuery du TD1. Si cela n'est pas fait, veuillez me contacter. Voici [l'interface](#) sur laquelle vous pouvez visualiser les données



The screenshot shows the Google Cloud BigQuery interface. On the left, the 'Explorer' pane shows the project 'ai-technologies-ur2' with a dataset 'dataset_teacher' containing a table named 'events'. The main pane displays the 'events' table schema with the following fields:

Field name	Type	Mode	Key	Collation	Default Value	Policy Tags	Description
city	STRING	NULLABLE	-	-	-	-	-
artist_image_src	STRING	NULLABLE	-	-	-	-	-
properly_sized_image_url	STRING	NULLABLE	-	-	-	-	-
call_to_action_redirect_url	STRING	NULLABLE	-	-	-	-	-
fallback_image_url	STRING	NULLABLE	-	-	-	-	-
artist_name	STRING	NULLABLE	-	-	-	-	-
venue_name	STRING	NULLABLE	-	-	-	-	-
streaming_event	BOOLEAN	NULLABLE	-	-	-	-	-
title	STRING	NULLABLE	-	-	-	-	-
location_text	STRING	NULLABLE	-	-	-	-	-
pin_icon_src	STRING	NULLABLE	-	-	-	-	-
event_url	STRING	NULLABLE	-	-	-	-	-
artist_url	STRING	NULLABLE	-	-	-	-	-
watch_live_text	STRING	NULLABLE	-	-	-	-	-
is_plus	BOOLEAN	NULLABLE	-	-	-	-	-
call_to_action_text	STRING	NULLABLE	-	-	-	-	-
rsvp_count	STRING	NULLABLE	-	-	-	-	-
rsvp_count_int	INTEGER	NULLABLE	-	-	-	-	-
starts_at	STRING	NULLABLE	-	-	-	-	-
ends_at	STRING	NULLABLE	-	-	-	-	-
timezone	STRING	NULLABLE	-	-	-	-	-

3. Rappel sur le service account

Lors du TD1 je vous ai fourni une clef JSON liée à un service account. Ce service account est un compte qui permet d'effectuer certaines opérations sur GCP. C'est une bonne pratique d'interagir avec un compte de service et non pas directement votre utilisateur@google.com. Dans le cadre de ce cours je vous donnerais néanmoins les droits de requêter vos données directement sans utiliser ce service account. Vérifier que vous pouvez visualiser votre table de données sur BigQuery.

4. Installer Fast API

Fast API est un framework Python rapide et efficace pour créer des API performantes et faciles à maintenir. En utilisant les annotations typées de Python, il garantit la validation des données en entrée et génère automatiquement une documentation interactive. C'est un choix idéal pour exposer des modèles de prédiction ou des analyses de données.

- **FastAPI** offre une architecture asynchrone et un système de validation robuste, parfait pour des applications de machine learning et de data science.

Installation

Je vous conseille d'utiliser un environnement virtuel pour gérer vos dépendances comme le fait poetry (optionnel).

Installer Fast API en suivant [la documentation](#). Vérifier l'installation de fastapi et de uvicorn

```
fastapi -version
uvicorn -version
```

Fast API et Uvicorn

- FastAPI ne fonctionne pas seul : il utilise un serveur ASGI comme **Uvicorn** pour servir l'application;

ASGI (Asynchronous Server Gateway Interface) est une interface qui gère les communications entre le serveur web et l'application FastAPI. Elle permet de gérer plusieurs requêtes en parallèle, grâce à son fonctionnement asynchrone. Contrairement à WSGI, ASGI est conçu pour traiter des tâches simultanées, ce qui le rend idéal pour des applications rapides et performantes comme FastAPI.

Schéma d'une requête API avec FastAPI et ASGI

1. Requête de l'utilisateur : L'utilisateur envoie une requête vers l'API (ex. : /events).
2. Serveur ASGI : Le serveur (comme Uvicorn) reçoit la requête et la redirige vers FastAPI, en permettant de traiter plusieurs requêtes en même temps.
3. FastAPI : routage et validation :
 - a. Routage : FastAPI identifie quel endpoint (chemin de l'URL) va traiter la requête.
 - b. Validation des données : FastAPI vérifie que les données de la requête respectent les types attendus.
4. Traitement de la requête : L'API traite la requête, récupère des données (par ex., dans BigQuery), ou exécute une prédiction, puis génère une réponse.

5. Réponse envoyée : FastAPI renvoie la réponse (souvent au format JSON) au serveur ASGI, qui l'envoie à l'utilisateur.
6. Résultat : L'utilisateur reçoit et utilise la réponse dans son application.

Ce processus permet à FastAPI, via ASGI, de répondre rapidement et efficacement aux utilisateurs en traitant les requêtes simultanément.

5. Vos premiers endpoints

1. Créer deux dossiers `api/app/`
2. Dans `app/` ajouter un fichier `api/app/main.py` contenant ce code

```
from typing import Union

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"Hello": "World"}
```

`from typing import Union`

`typing` est un module contenant des objets représentant tous les types en Python.

Le typage est très important en programmation car il permet aux développeurs de mieux comprendre le code mais surtout d'accélérer certaines fonctions. Malheureusement Python n'est pas un langage stricte sur le typage. C'est en partie pourquoi il est plus flexible mais plus lent que d'autres. Fast API utilise une librairie Python qui va vérifier le type de chaque argument. C'est donc obligatoire de typer vos arguments car s'ils n'ont pas le bon format Fast API renverra une erreur.

`app = FastAPI()`

`app` est une instance de FastAPI qui représente votre application. C'est grâce à cette instance que vous allez définir les routes (ou endpoints) et configurer votre API. Elle est essentielle, car elle permet à FastAPI de "savoir" quelles fonctions exécuter en fonction des requêtes reçues

```
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

- `@app.get("/")` : Ce décorateur indique que cette fonction est liée à l'endpoint racine (`/`) de l'API et que la méthode HTTP est `GET`. Quand quelqu'un visite l'URL de l'API suivie d'un simple `/`, cette fonction est exécutée.
- `def read_root()` : Cette fonction est appelée `read_root`, mais elle pourrait avoir n'importe quel nom. Elle ne prend pas de paramètres car elle ne nécessite aucune donnée de l'utilisateur.
- `return {"Hello": "World"}` : La fonction renvoie un dictionnaire Python qui sera automatiquement converti en `JSON` pour la réponse API. Ici, la réponse contient simplement le message "Hello": "World".

3. Lancer `fastapi dev api/app/main.py`

4. Visualiser votre localhost



- Ici votre `localhost` est le lien de votre FAST API pour le développement en local. Vous pouvez voir qu'aller sur le lien racine a ensuite exécuté la fonction `read_root` et vous renvoie le dictionnaire Python

5. Sur votre navigateur essayer de naviguer jusqu'à `http://127.0.0.1:8000/mon_endpoint`

Vous devriez voir que l'instance ne trouve rien

6. Retourner dans votre code et ajouter l'endpoint suivant

```
@app.get("/mon_endpoint/{objet_id}")
async def read_objet(objet_id: int, q: Union[str, None] = None):
    return {"objet_id": objet_id, "q": q
```

On crée ici un endpoint GET qui se déclenche quand on navigue sur `/mon_endpoint`. `objet_id` est une partie dynamique de l'URL ; chaque valeur passée à cet endroit sera capturée comme argument `objet_id` de la fonction `read_objet`.

En ce qui concerne le typage `Union[str, None]` veut dire que l'argument peut être soit une string soit être `None`. On aurait pu également utiliser `Optional[str]` ou `str | None`

```
def read_objet(objet_id: int, q: Union[str, None] = None):
```

La fonction `read_objet` prend deux arguments :

`objet_id` est un paramètre obligatoire de type entier, capturé dans l'URL.

`q` est un paramètre optionnel qui peut être soit une chaîne (str), soit `None`. Si `q` n'est pas spécifié dans la requête, il prend la valeur `None` par défaut.

`return {"objet_id": objet_id, "q": q}` : La fonction renvoie un dictionnaire contenant `objet_id` et `q`, qui sera converti en JSON pour la réponse.

7. Naviguer jusqu'à http://127.0.0.1:8000/mon_endpoint/1

Vous devriez voir que l'application vous renvoie bien un dictionnaire avec `objet_id = 1` et `q = None`

Maintenant ajouter le paramètre `q` qui était optionnel.

Puis remplacer 1 par une string, que se passe t-il ?

output : La réponse de l'application avec le parametre optionnel + la réponse quand le type de `objet_id` n'est pas un integer

6. Endpoint PUT

1.

```
from pydantic import BaseModel

class Objet(BaseModel):
    name: str
    price: float
    is_offer: Union[bool, None] = None
```

```
@app.put("/mon_endpoint/{objet_id}")
def update_objet(objet_id: int, objet: Objet):
    return {"objet_name": objet.name, "objet_id": objet_id}
```

`@app.put("/mon_endpoint/{objet_id}") :`

Ce décorateur indique que ce endpoint accepte les requêtes **PUT** à l'URL

`/mon_endpoint/{objet_id}`. La méthode **PUT** est souvent utilisée pour mettre à jour des ressources existantes.

```
def update_objet(objet_id: int, objet: Objet):
```

La fonction `update_objet` prend deux paramètres :

`objet_id` est un paramètre d'URL, capturé dans l'URL et de type `int`.

`objet` est un objet de type `Objet`, le modèle de données défini plus tôt. **FastAPI** extrait automatiquement les données du corps de la requête et les valide par rapport au modèle `Objet`.

```
return {"objet_name": objet.name, "objet_id": objet_id}
```

La fonction renvoie un dictionnaire contenant le nom de l'objet (`objet.name`) et son identifiant (`objet_id`). Ce dictionnaire sera converti en JSON et envoyé comme réponse.

2. Naviguez jusqu'à <http://127.0.0.1:8000/docs>

Cette page est créée automatiquement par FAST API et vous permet d'interagir manuellement avec vos endpoints.

output : Interagir avec vos 3 endpoints grâce à la pages /docs

7. Préparer votre architecture de code

Lisez ce [blog](#) pour comprendre comment vous allez structurer votre code dans votre GIT repository

II - Construire votre API

Vous êtes désormais prêts à créer votre API ! Votre mission consiste à concevoir au moins 8 endpoints. Chaque endpoint doit être pensé pour répondre aux besoins d'un public précis. Lors de la dernière session, vous présenterez votre API à l'oral en adoptant une démarche de "vente", en ciblant soit un public spécifique, soit un public plus général. Veillez donc à créer des endpoints qui apportent une réelle valeur.

Je vous impose la création des 3 endpoints.

1. S'authentifier à BigQuery depuis votre API

Vous allez devoir vous authentifier à BigQuery depuis votre code qui est exécuté par votre machine local. Reprenez le TD1 et créer une fonction python utilitaire permettant de récupérer des données précises en utilisant votre service account. Cette fonction pourrait exécuter une requête sql sur votre table BigQuery

- [Doc](#)
- [Doc](#)

Il est obligatoire d'utiliser le compte de service pour requêter vos données. Imaginez une seconde déployer votre API en production, vous ne pourrez pas alors requêter vos données depuis votre utilisateur perso

output : Fonctions utilitaire pour se connecter et récupérer des données depuis votre dataset bigquery

2. Pagination

Pour garantir des performances optimales et des temps de réponse rapides, votre API implémentera la **pagination**. La pagination consiste à diviser les réponses volumineuses en plusieurs pages de taille réduite et facilement consultables. Cela permet aux utilisateurs de naviguer efficacement parmi les résultats sans surcharger le système.

Mise en œuvre de la pagination

Chaque requête à l'API doit inclure un paramètre **page** (par exemple, **?page=1**) afin de spécifier la page des résultats souhaitée.

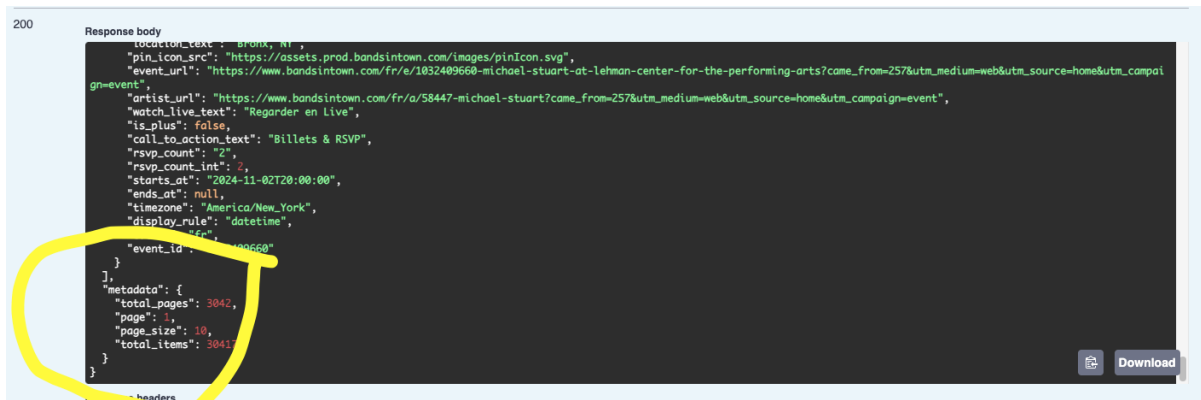
3. Metadata

Chaque endpoint aura une section "metadata" avec comme information :

- **page** : le numéro de la page
- **total_pages** : le nombre total de pages disponibles.
- **results_per_page** : le nombre d'éléments retournés par page.
- **total_results** : le nombre total d'éléments disponibles pour cette requête.
- **next_page_url** : l'url à requêter de la prochaine page s'il existe sinon **None**

4. Construction des endpoints obligatoires

Attention l'API doit pouvoir répondre rapidement. Pour se faire vous devez paginer les réponses. Il est donc nécessaire d'ajouter un paramètre page. De plus ajoutez une section metadata dans chaque réponse qui contient le nombre de page total que possède votre endpoint.



Endpoint de base pour tous les événements

- **Route** : **GET /events**
- **Description** : Retourne une liste de tous les événements pour la ville étudiée de votre base de données
- **Transformation** : Aucune.

Distribution des événements par jour de la semaine

- **Route** : `GET /events/by-day-of-week`
- **Arguments**
 - `week` : numéro de semaine dans une année. Si None alors pas de filtre sur la semaine précise
- **Description** : Retourne le nombre d'événements pour chaque jour de la semaine (lundi, mardi, mercredi ..).
- **Transformation** : `Group BY` sur le jour (`date` transformée en jour de la semaine) pour obtenir la distribution des événements. Filtre sur la semaine si indiquée.

Filtrer les événements par artiste et lieu

- **Route** : `GET /events/search`
- **Arguments** :
 - `artistName` (nom de l'artiste, optionnel)
 - `venueName` (nom du lieu, optionnel)
 - `date_range` (intervalle de dates, optionnel)
- **Description** : Retourne les événements en fonction de l'artiste, du lieu, et d'une période donnée.
- **Transformation** : Filtrage par `artistName`, `venueName`, et `date` entre les dates spécifiées.

Je vous conseille vivement de construire un endpoint lié à votre prédiction d'affluence/popularité construit lors du TD1. En effet cela ne pourra qu'appuyer votre oral pour distinguer votre API d'une API basique qui collecte et ressort les données.

output : Endpoints fonctionnels

5. Idées d'endpoint

Filtrer les événements par artiste

- **Route** : `GET /events/artist/{artistName}`
- **Description** : Retourne tous les événements associés à un artiste particulier.
- **Paramètre** : `artistName` (nom de l'artiste).
- **Transformation** : Filtre les événements en fonction de l'artiste. L'artiste doit pouvoir être rentré de plusieurs façons (Majuscule, faute d'orthographe, caractère spéciaux)

Statistiques RSVPs par venue

- **Route** : `GET /events/by-venue`

- **Description** : Retourne la moyenne, la mediane, le min, le max, le nombre d'événement par `venueName`.
- **Transformation** : `Group BY` sur `venueName`

Événements en fonction de la popularité

- **Route** : `GET /events/popularity`
- **Description** : Retourne les événements, classés par `Popularite` (ex : Très Populaire, Populaire, etc.).
- **Transformation** : Filtrage et classement des événements selon leur popularité.

Prévision d'affluence sur une période

- **Route** : `GET /events/predicted-attendance`
- **Description** : Prédit l'affluence (basé sur le modèle) pour une date spécifique.
- **Paramètre** : `date` (date ciblée pour la prédiction).

Événements en ligne vs. événements en présentiel

- **Route** : `GET /events/streaming-vs-in-person`
- **Description** : Retourne le nombre d'événements en ligne (`streamingEvent=True`) versus en présentiel.
- **Transformation** : `Group BY` sur `streamingEvent` avec comptage pour chaque type.

Liste des événements en fin de semaine

- **Route** : `GET /events/weekend`
- **Description** : Retourne tous les événements qui ont lieu durant le week-end.
- **Transformation** : Filtre les événements avec `Weekend=True`.

Statistiques RSVPs par type d'événement

- **Route** : `GET /events/average-rsvp-by-type`
- **Description** : Retourne les statistiques RSVP en fonction du `Type` de l'événement (par exemple, concert, streaming, etc.).
- **Transformation** : `Group BY` sur `Type`.

Durée moyenne des événements

- **Route** : `GET /events/average-duration`
- **Description** : Calcule la durée moyenne des événements.
- **Transformation** : Calcul de la durée moyenne (`duree`).

Événements en approche (dans les 7 jours)

- **Route** : `GET /events/upcoming`
- **Description** : Retourne les événements qui auront lieu dans les 7 prochains jours.

- **Transformation** : Filtrage par `Nb_jours_avant < 7`.

Évolution du RSVP au fil du temps (Rolling Window)

- **Route** : `GET /events/rsvp-trend`
- **Description** : Montre la tendance des RSVPs en utilisant une moyenne glissante de 7 jours.
- **Transformation** : Rolling Window sur 7 jours pour `rsvpCount`.

Top 5 des artistes les plus populaires

- **Route** : `GET /events/top-artists`
- **Description** : Retourne les 5 artistes ayant le plus d'événements ou de RSVPs.
- **Transformation** : Classement des artistes en fonction du nombre d'événements ou de RSVPs.

Nombre d'événements par fuseau horaire

- **Route** : `GET /events/by-timezone`
- **Description** : Retourne le nombre d'événements par fuseau horaire (`timezone`).
- **Transformation** : `Group BY` sur `timezone`.

6. Endpoint Bonus

Construisez un endpoint de type PUT qui modifie les données dans votre dataset BigQuery

Mettre à jour le nom de l'artiste pour un événement spécifique

- **Route** : `PUT /events/{event_id}/update-artist`
- **Description** : Met à jour le nom de l'artiste pour un événement donné.

Modifier le lieu (venue) d'un événement

- **Route** : `PUT /events/{event_id}/update-venue`
- **Description** : Change le nom du lieu (`venueName`) pour un événement spécifique.

output : Endpoint de type PUT

III - Sécuriser votre API

Il est important de protéger votre API pour éviter que n'importe qui puisse l'utiliser sans autorisation. Sans sécurité, des utilisateurs non autorisés pourraient accéder aux données, surcharger votre serveur avec des requêtes, ou même modifier des informations sensibles.

Pour une protection simple, nous utilisons un **token d'authentification**. Un token agit comme un mot de passe : seuls ceux qui possèdent ce token peuvent faire des requêtes sur l'API.

1. Créer votre fonction de vérification

Pour se faire vous allez créer une fonction python qui vérifie que le token utilisé par l'utilisateur est légitime. Cette fonction renverra une réponse 403 si le token n'est pas légitime. Voici la [nomenclature](#) des réponses HTTP

[Voici](#) comment faire si votre token de sécurité est passé dans les Headers (En-tête) de votre endpoint. Attention /docs ne vous permettra pas de passer des Headers pour tester votre API vous devrez alors requêter via du python ou bash (en injectant votre token dans les en tête)

Un autre moyen serait de vérifier le token en exigeant ce token en tant que paramètre de la requête. Cela est moins sécurisé mais dans le cadre de ce cours cette solution sera acceptée.

2. Appliquer la sécurité

Appliquer cette sécurité partout, **il ne faut en aucun cas que le ou les tokens légitimes soient écrits en dur dans le code**. Cela se retrouverait alors dans votre repo git ou pire dans les requêtes réseaux effectuées par les utilisateurs. Les mots de passe de base de données, de service account etc ne doivent jamais être en dur dans le code c'est l'une des règles les plus importantes lors d'entretien de développeur.

output : Une API sécurisée

TD Oral (3H)

Pour votre présentation finale, l'objectif est de convaincre votre audience de la valeur de votre API. Choisissez un public cible pour lequel votre API est particulièrement adaptée (par exemple : grandes entreprises, particuliers, intégration open source, etc.).

Expliquez clairement l'utilité de votre API, les données que vous utilisez, et pourquoi cela pourrait être intéressant pour votre public choisi. Les autres groupes joueront le rôle de l'audience, donc n'oubliez pas que vous devez vendre votre API et susciter l'intérêt. Adoptez une approche convaincante, en vous appuyant sur un produit bien conçu et une explication claire de ses fonctionnalités. Pourquoi pas une démo de votre API ? une mise en contexte ..

Votre présentation ne devra pas dépasser 12 minutes, suivie de 3 minutes de questions.